Count-Min Sketch

A Probabilistic Data Structure

12 October, 2026



Team Members: Anmol Sriwastava, Shinjan Majumdar, Md. Danish, Ankesh Raj, Pardha Saradhi

Probability II

The Frequency Estimation Problem

Given a stream $x_1, x_2, x_3, \ldots$ arriving one at a time, estimate $f(x)$, how many times x has appeared so far.

Streams:

- Spotify song plays
- Router packets (by IP)
- Twitter hashtags
- Ad clicks
- Google searches

Constraints:

- Items arrive one at a time
- Cannot store the stream
- Single pass, no rewinding
- Memory is limited
- Data is too fast to store

The Tension

Exact counts require storing **every distinct item** seen so far. At scale, this is **prohibitively expensive**.

One might think: is there actually a need for such a data structure?

The answer is Yes.

Let us see with the help of an example.

Let us try to solve this frequency estimation problem using a **MultiSet** data structure.

Attempt: MultiSet (Hash Map)

Idea

Store each distinct item as a key. Increment its count every time it appears.

Incoming stream:

```
192.170.0.1
75.245.10.1
10.125.22.20
192.170.0.1
⋮
```

MultiSet state after 4 inserts:

Key (IP address)	Count
192.170.0.1	2
75.245.10.1	1
10.125.22.20	1

Query: 192.170.0.1 $\rightarrow$ **2**

But this scales poorly...

Every **distinct** item adds a new row. One row per unique IP.

The Cost of MultiSet at Scale

Insertion Time

# UUIDs	Time (ms)
10	< 25
100	< 25
1,000	30
10,000	257
100,000	1200
1,000,000	4244

Memory (JVM Heap)

# UUIDs	Heap (MB)
10	< 2
100	< 2
1,000	3
10,000	9
100,000	39
1,000,000	234

Observation

Both time and memory grow **linearly** with the number of distinct items. At 1M items: **4.2 seconds** and **234 MB**. At a billion: **impractical**.

Hashing as a Black Box: Mechanics

What a hash function does

A hash function h maps a key to a bucket index:

$$h(k) \;=\; k \bmod m.$$

k – Key

The input value.
Any non-negative integer.

Examples: an IP address, a song ID, a hashtag.

m – Table size

The number of slots (buckets).

Usually chosen to be a prime number to spread keys more evenly.

$h(k)$ – Hash value

The index where k is stored.

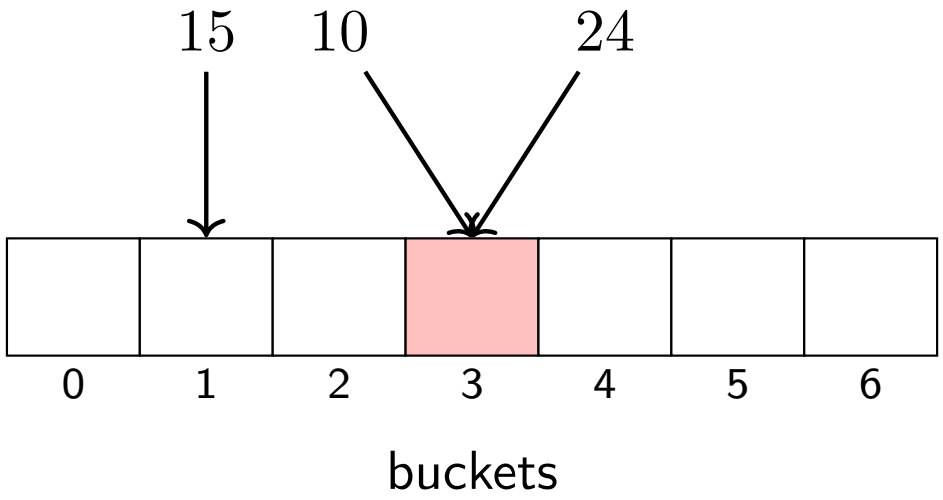
An integer in $\{0, 1, \ldots, m-1\}$.

Key	Hash function	Bucket
15	$h(15) = 15 \bmod 7$	1
10	$h(10) = 10 \bmod 7$	3
24	$h(24) = 24 \bmod 7$	3

Keys 10 and 24 map to the same bucket – a **collision**.

Collisions

- There are far more possible keys than buckets, so different keys must sometimes land in the *same* bucket. This is called a collision.



Keys 10 and 24 are sent to the same bucket by h: a collision (shaded).

Hashing as a Black Box: Assumptions

We never look inside h

From this point on, h is treated as a black box. We rely only on three statistical properties.

1. Uniformity.
Each item is equally likely to land in any of the w cells:

$$\Pr\big[h(k) = j\big] \;=\; \frac{1}{w} \qquad \text{for every } j \in \{1, \ldots, w\}.$$

2. Pairwise independence.
For any two distinct items $x \neq y$:

$$\Pr\big[h(x) = h(y)\big] \;\leq\; \frac{1}{w}.$$

3. Independent rows.
The d hash functions $h_1, h_2, \ldots, h_d$ are chosen independently of each other. A collision in row i tells us nothing about a collision in row j.

Count-Min Sketch: The Idea

Using multiple hash functions

A hash function h maps the huge set of possible items to a much smaller set of w columns $\{1, \ldots, w\}$. Such functions can be constructed so that, for distinct items $x_1, x_2, \ldots,$

$$h(x_1),\ h(x_2),\ \ldots \overset{\text{iid}}{\sim} \text{Unif}\{1, \ldots, w\}.$$

Moreover, one can construct several functions $h_1, h_2, \ldots, h_d$ (one per row) whose outputs behave like independent random variables.

- The randomness lies in the **choice** of h, made once at the start. Afterwards h is a fixed, deterministic function.
- All probabilities are taken over the random choice of $h_1, \ldots, h_d$; the stream itself is fixed.

`192.170.0.1` $\longrightarrow$

w columns							
0	0	0	**1**	0	0	0	0
0	0	0	0	0	0	**1**	0
0	**1**	0	0	0	0	0	0

$d = 3$ hash functions, each row = one hash

Count-Min Sketch: The Two Operations

Update(x)

Pass x through all d hash functions.

Increment the cell in each row at the returned column.

$$
\begin{aligned}
h_1(x) &\rightarrow \texttt{matrix[1][}\cdot\texttt{]++} \\
h_2(x) &\rightarrow \texttt{matrix[2][}\cdot\texttt{]++} \\
&\vdots \\
h_d(x) &\rightarrow \texttt{matrix[d][}\cdot\texttt{]++}
\end{aligned}
$$

Query(x)

Pass x through all d hash functions.

Read the cell in each row. Return the **minimum**.

$$
\begin{aligned}
h_1(x) &\rightarrow \texttt{matrix[1][}\cdot\texttt{]} \\
h_2(x) &\rightarrow \texttt{matrix[2][}\cdot\texttt{]} \\
&\vdots \\
h_d(x) &\rightarrow \texttt{matrix[d][}\cdot\texttt{]} \\
&\quad \textbf{return } \min(\ldots)
\end{aligned}
$$

Why minimum?

Each cell is an **upper bound** on the true frequency (other items may collide and inflate it). The minimum across rows gives the **tightest** bound.

Expected Collision Noise

Song play counts

Song	Plays
A	40
B	20
C	30
D	10

One hash row: $w = 4$ **buckets**

Bucket	Counter
0	0
1	0
2	$40 + 20 = 60$
3	0

$$h(A) = h(B) = 2$$

$$N = 100, \qquad f(A) = 40$$

Query: estimate the plays of song A.

$$\underbrace{60}_{\text{Estimate}} = \underbrace{40}_{\text{True count}} + \underbrace{20}_{\text{Collision noise}}$$

Mathematical Model

$$X_i = \sum_{y \neq A} f(y) I_{i,y}, \qquad I_{i,y} = \mathbf{1}\{h_i(y) = h_i(A)\}$$

Expected Collision Noise: Collision Probability

Suppose there are w buckets

0	1	2	3

$$\Pr[h_i(B) = k] = \frac{1}{w}, \qquad \Pr[h_i(A) = k] = \frac{1}{w}$$

For both songs to land in the same bucket, they must coincide in one of the w possible buckets.

$$\begin{aligned}\Pr[h_i(B) = h_i(A)] &= \sum_{k=1}^{w} \Pr[h_i(B) = k] \Pr[h_i(A) = k] \\ &= \sum_{k=1}^{w} \frac{1}{w} \cdot \frac{1}{w} \\ &= \boxed{\frac{1}{w}}\end{aligned}$$

Indicator Variable

$$I_{i,B} = \begin{cases} 1 & h_i(B) = h_i(A), \\ 0 & \text{otherwise} \end{cases} \qquad \boxed{\mathbb{E}[I_{i,B}] = \frac{1}{w}}$$

Expected Collision Noise: Taking Expectation

For a query on song A:

$$X_i = \sum_{y \neq A} f(y) I_{i,y}$$

Using linearity of expectation and $\mathbb{E}[I_{i,y}] = 1/w$:

$$\begin{aligned}
\mathbb{E}[X_i] &= \sum_{y \neq A} f(y) \mathbb{E}[I_{i,y}] \\
&= \frac{1}{w} \sum_{y \neq A} f(y) \\
&= \frac{N - f(A)}{w} \\
&\leq \boxed{\frac{N}{w}}
\end{aligned}$$

N total plays $\longrightarrow$ N/w

Expected-noise bound

Markov Inequality: Bounding One Row

Known result

$$\mathbb{E}[X_i] \leq \frac{N}{w}$$

We want to bound the probability that the collision noise exceeds the allowed error:

$$t = \varepsilon N$$

Markov's inequality

For $X_i \geq 0$ and $t > 0$:

$$\Pr[X_i \geq t] \leq \frac{\mathbb{E}[X_i]}{t}$$

Substituting $t = \varepsilon N$:

$$\begin{aligned} \Pr[X_i \geq \varepsilon N] &\leq \frac{\mathbb{E}[X_i]}{\varepsilon N} \\ &\leq \frac{N/w}{\varepsilon N} \\ &= \boxed{\frac{1}{\varepsilon w}} \end{aligned}$$

Result

The probability of excessive collision noise in one row is at most $1/(\varepsilon w)$.

Combining Across Rows: Why the Minimum Works

Three independent hash functions

	0	1	2	3	Count for A
h_1	0	0	60	0	60
h_2	0	40	0	0	40
h_3	0	0	50	0	50

$$\hat{f}(A) = \min(60, 40, 50) = 40$$

$$\underbrace{40}_{\text{True frequency}} \leq \underbrace{\hat{f}(A)}_{\text{Estimate}}$$

Probability of Excessive Error

$$\Pr[\hat{f}(A) > f(A) + \varepsilon N] = \Pr[X_1 > \varepsilon N, \ldots, X_d > \varepsilon N]$$

$$\leq \boxed{\left(\frac{1}{\varepsilon w}\right)^d}$$

The (ε, δ) Guarantee

Choose the width

$$\boxed{w = \left\lceil \frac{e}{\varepsilon} \right\rceil}$$

Therefore,

$$\frac{1}{\varepsilon w} \leq \frac{1}{e}$$

Choose the depth

$$\boxed{d = \left\lceil \ln \frac{1}{\delta} \right\rceil}$$

Failure probability

$$\begin{aligned} \Pr[\text{excessive error}] &\leq \left(\frac{1}{\varepsilon w} \right)^d \\ &\leq e^{-d} \\ &\leq \delta \end{aligned}$$

Hence,

$$\boxed{\Pr[\text{success}] \geq 1 - \delta}$$

Final Result: For a Fixed Song x

$$\boxed{f(x) \leq \hat{f}(x) \leq f(x) + \varepsilon N}$$

with probability at least $1 - \delta$.

Let's see it with help of an example.

A Spotify-style simulation of the Count-Min Sketch

[Launch Simulation]

Advantages of the Count-Min Sketch

- **Fixed memory**
  $d \times w$ counters,
  independent of N
- **Constant-time operations**
  Update: $O(d)$ Query: $O(d)$
- **Single pass, streaming-native**
  Never revisit an event

Memory never grows

$d \times w$ counters			
1	1	1	1
1	1	1	1
1	1	1	1

fixed, no matter how many events flow through

Advantages of the Count-Min Sketch

- **Never underestimates**

  $\hat{f}(x) \geq f(x)$ always

- **Provably bounded error**

  Within εN with probability $1 - \delta$

Time is flat

Events	10	10^3	10^5	10^7
CMS	$O(d)$	$O(d)$	$O(d)$	$O(d)$
Map	$O(1)$	$O(1)$	$O(1)$	$O(\log n)$

CMS time is **constant** regardless of stream size

The estimate is always **from above**: the truth, plus at most εN.

Disadvantages of the Count-Min Sketch

- **Overestimates only**
  Collisions inflate counts; never deflate
- **No deletions**
  Standard CMS is insert-only
- **Probabilistic guarantee**
  Bounded error w.p. $1 - \delta$, not certain
- **Cannot list items**
  Answers "how many," not "which ones"
- **Parameter tuning**
  Must choose ε, δ before the stream

One-sided error

Estimate	Truth
$\hat{f}(x)$	$f(x)$
$\geq$	always

Insert-only

Event	Action	Possible?
insert x	$+1$	Yes
delete x	?	No

No enumeration

"What is $f(x)$?" **Yes** "Which x have $f \geq k$?" **No**

Thank you.

Questions?